

File System Operations — The `fs` and `path` Modules

Section: 5 — File System Operations and Streams

Subtopic: 01 — File System Module

Estimated Duration: 3–4 hours

Theory

Practical

Topics covered: `fs` module basics (`readFile` , `writeFile` , `appendFile` — sync and async variants), the `path` module (`join` , `resolve` , `extname` , `basename` , `dirname`), directory operations (`mkdir` , `readdir` , `rmdir` / `rm`)

Learning Objectives — This Subtopic

- Read, write, and append files using both synchronous and asynchronous `fs` methods
- Explain the trade-offs between sync and async file operations and choose the appropriate variant
- Construct cross-platform file paths using the `path` module instead of string concatenation
- Create, list, and remove directories programmatically
- Handle common file system errors (`ENOENT` , `EEXIST` , `EACCES`) gracefully
- Use the promises API (`fs/promises`) with `async/await` for clean asynchronous file operations

1. Why File System Access Matters in Node.js

In the browser, JavaScript has no direct access to the user's file system — this is a deliberate security restriction. Node.js removes this limitation. Server-side applications routinely need to read configuration files, write log entries, process uploaded documents, generate reports, and manage directory structures.

Node.js provides the `fs` (file system) module as part of its core library. No installation is required. It offers three flavours of every operation:

Flavour	Example	Blocks Event Loop?	When to Use
Synchronous	<code>fs.readFileSync()</code>	Yes	Scripts, CLI tools, application startup
Callback	<code>fs.readFile()</code>	No	Traditional async pattern
Promise	<code>fsPromises.readFile()</code>	No	Modern async/await code

Key Insight: From Section 3 (Event Loop), recall that synchronous operations block the event loop. In a server handling many concurrent requests, a single `readFileSync()` call freezes *all* other work until the file read completes. Use synchronous methods only at startup or in single-run scripts, never inside request handlers.

Set up a project folder for this section's exercises:

```
mkdir fs-basics && cd fs-basics
```

```
npm init -y
```

2. Reading Files

2.1 Synchronous Read — `fs.readFileSync()`

Start with the simplest case. Create a text file to work with:

```
message.txt
```

```
Hello from the file system.  
This is line two.  
And this is line three.
```

Now read it synchronously:

`read-sync.js`

```
const fs = require('fs');  
  
const content = fs.readFileSync('message.txt', 'utf8');  
  
console.log(content);  
console.log('Type:', typeof content);
```

`node read-sync.js`

Output:

```
Hello from the file system.  
This is line two.  
And this is line three.  
Type: string
```

What is happening?

- `fs.readFileSync(path, encoding)` reads the entire file into memory and returns its contents.
- The second argument, `'utf8'`, tells Node.js to decode the raw bytes as a UTF-8 string. Without it, the return value is a `Buffer` object (raw binary data) rather than a string.
- Execution pauses on the `readFileSync` line until the read completes — this is blocking behaviour.

2.2 What Happens Without an Encoding?

`read-buffer.js`

```
const fs = require('fs');  
  
const raw = fs.readFileSync('message.txt');
```

```
console.log(raw);
console.log('Type:', typeof raw);
console.log('Is Buffer:', Buffer.isBuffer(raw));
console.log('As string:', raw.toString('utf8'));
```

```
node read-buffer.js
```

Output:

```
<Buffer 48 65 6c 6c 6f 20 66 72 6f 6d 20 74 68 65 20 66 69 6c 65 20 73 79 73 74 65 6d 2e 0a 54 68 69 73 20 69
73 20 6c 69 6e 65 20 74 77 6f 2e 0a 41 6e 64 ... 24 more bytes>
Type: object
Is Buffer: true
As string: Hello from the file system.
This is line two.
And this is line three.
```

The `Buffer` contains the raw bytes. Each hex pair (`48` , `65` , `6c` , etc.) is one byte. `48` is the ASCII code for `H` , `65` for `e` , and so on. Calling `.toString('utf8')` converts the Buffer back to a readable string.

Note: When reading text files (source code, configuration, logs), always pass the encoding. When reading binary files (images, PDFs, executables), omit the encoding and work with the Buffer directly.

2.3 Asynchronous Read — Callback Style

The callback-based `fs.readFile()` follows the error-first callback convention covered in Section 3:

```
read-callback.js
```

```
const fs = require('fs');

console.log('Before read');

fs.readFile('message.txt', 'utf8', (err, data) => {
  if (err) {
    console.error('Error reading file:', err.message);
  }
});
```

```
    return;
  }
  console.log('File contents:', data);
});

console.log('After read (this runs first)');
```

```
node read-callback.js
```

Output:

```
Before read
After read (this runs first)
File contents: Hello from the file system.
This is line two.
And this is line three.
```

What is happening?

- `fs.readFile()` returns immediately. The actual file read happens in the background (via libuv's thread pool).
- The callback fires once the read is complete. "After read" prints *before* the file contents because the synchronous `console.log` runs first, while the file I/O is still in progress.
- The callback receives `(err, data)`. If the read succeeds, `err` is `null` and `data` contains the file contents. If it fails, `err` contains an Error object.

2.4 Asynchronous Read — Promises with `async/await`

The promises API is available via `require('fs/promises')` (or `require('fs').promises`). Combined with `async/await` from Section 3, this gives the cleanest syntax:

```
read-async.js
```

```
const fs = require('fs/promises');

async function readMessage() {
  try {
    const content = await fs.readFile('message.txt', 'utf8');
    console.log('File contents:', content);
  } catch (err) {
```

```
        console.error('Error reading file:', err.message);
    }
}

readMessage();
```

```
node read-async.js
```

Output:

```
File contents: Hello from the file system.
This is line two.
And this is line three.
```

This approach is non-blocking (like the callback version) but reads linearly from top to bottom. It is the recommended pattern for most modern Node.js code.

Important: `require('fs/promises')` and `require('fs')` are different modules. The promises module does *not* contain the synchronous methods. If you need both, import them separately:

```
const fs = require('fs');           // sync + callback
const fsPromises = require('fs/promises'); // promise-based
```

3. Writing Files

3.1 `fs.writeFileSync()` / `fs.writeFile()`

`writeFile` creates a new file or **completely replaces** the contents of an existing file. This is a destructive operation.

Stage 1 Synchronous write:

```
write-sync.js
```

```
const fs = require('fs');

fs.writeFileSync('output.txt', 'This file was created by Node.js.\n');

console.log('File written successfully');

// Verify by reading it back
const content = fs.readFileSync('output.txt', 'utf8');
console.log('Contents:', content);
```

```
node write-sync.js
```

Output:

```
File written successfully
Contents: This file was created by Node.js.
```

Stage 2 Promise-based write with `async/await` :

```
write-async.js
```

```
const fs = require('fs/promises');

async function writeGreeting() {
  const timestamp = new Date().toISOString();
  const content = `Generated at: ${timestamp}\nStatus: operational\n`;

  await fs.writeFile('status.txt', content, 'utf8');
  console.log('status.txt written');

  // Verify
  const result = await fs.readFile('status.txt', 'utf8');
  console.log(result);
}

writeGreeting();
```

```
node write-async.js
```

Output:

```
status.txt written
Generated at: 2025-02-25T10:30:00.000Z
Status: operational
```

Warning: `writeFile` silently overwrites existing content. If `output.txt` already contained 500 lines of data, calling `writeFile` replaces it entirely. If you want to *add* to a file, use `appendFile` instead.

3.2 Writing JSON Data

A common task is writing JavaScript objects to disk as JSON. `writeFile` only accepts strings or Buffers, so use `JSON.stringify()` first:

`write-json.js`

```
const fs = require('fs/promises');

async function saveConfig() {
  const config = {
    appName: 'MyApp',
    port: 3000,
    debug: false,
    allowedOrigins: ['http://localhost:3000', 'https://example.com']
  };

  // The third argument (2) adds indentation for readability
  const json = JSON.stringify(config, null, 2);

  await fs.writeFile('config.json', json, 'utf8');
  console.log('Config saved to config.json');

  // Read it back and parse
  const raw = await fs.readFile('config.json', 'utf8');
  const parsed = JSON.parse(raw);
  console.log('App name:', parsed.appName);
  console.log('Port:', parsed.port);
}

saveConfig();
```

```
node write-json.js
```


Output:

Config saved to config.json
App name: MyApp
Port: 3000

The generated `config.json` looks like this:

config.json

```
{
  "appName": "MyApp",
  "port": 3000,
  "debug": false,
  "allowedOrigins": [
    "http://localhost:3000",
    "https://example.com"
  ]
}
```

4. Appending to Files

`appendFile` adds content to the *end* of an existing file. If the file does not exist, it creates it. Unlike `writeFile`, existing content is preserved.

append-demo.js

```
const fs = require('fs/promises');

async function logActivity() {
  const logFile = 'activity.log';

  // Each call adds a line – nothing is overwritten
  await fs.appendFile(logFile, `[${new Date().toISOString()}] Server started\n`);
  await fs.appendFile(logFile, `[${new Date().toISOString()}] User login: alice\n`);
  await fs.appendFile(logFile, `[${new Date().toISOString()}] User login: bob\n`);

  console.log('Log entries written');
```

```
const log = await fs.readFile(logFile, 'utf8');
console.log(log);
}

logActivity();
```

```
node append-demo.js
```

Output:

```
Log entries written
[2025-02-25T10:30:01.000Z] Server started
[2025-02-25T10:30:01.001Z] User login: alice
[2025-02-25T10:30:01.002Z] User login: bob
```

Run the script again and the log grows — previous entries remain intact:

```
node append-demo.js
```

Output:

```
Log entries written
[2025-02-25T10:30:01.000Z] Server started
[2025-02-25T10:30:01.001Z] User login: alice
[2025-02-25T10:30:01.002Z] User login: bob
[2025-02-25T10:31:05.000Z] Server started
[2025-02-25T10:31:05.001Z] User login: alice
[2025-02-25T10:31:05.002Z] User login: bob
```

Key Insight: `appendFile` is the right choice for log files, audit trails, and any scenario where you need to accumulate data over time. `writeFile` is for creating or replacing files from scratch.

Quick Reference — Read / Write / Append

Operation	Sync	Callback	Promise	Behaviour
Read	<code>readFileSync()</code>	<code>readFile()</code>	<code>fsPromises.readFile()</code>	Returns file contents

Operation	Sync	Callback	Promise	Behaviour
Write	<code>writeFileSync()</code>	<code>writeFile()</code>	<code>fsPromises.writeFile()</code>	Creates or replaces
Append	<code>appendFileSync()</code>	<code>appendFile()</code>	<code>fsPromises.appendFile()</code>	Creates or adds to end

5. The `path` Module — Building Paths Safely

Hard-coding file paths with forward slashes (/) works on macOS and Linux but breaks on Windows, which uses backslashes (\). The built-in `path` module constructs correct paths regardless of operating system.

`path-demo.js`

```
const path = require('path');
```

Analogy: Think of `path` as a GPS for the file system. Instead of manually typing an address and hoping you got the country-specific format right, you give it the components (city, street, house number) and it assembles the correct address for wherever you are.

5.1 `path.join()` — Combining Path Segments

`path.join()` concatenates path segments using the platform-specific separator and normalises the result:

`path-join.js`

```
const path = require('path');

const filePath = path.join('users', 'alice', 'documents', 'report.txt');
console.log(filePath);
```

```
// It handles extra slashes and .. segments
const cleaned = path.join('users', '//alice', '../bob', 'notes.txt');
console.log(cleaned);
```

```
node path-join.js
```

Output:

```
(On macOS/Linux:)
users/alice/documents/report.txt
users/bob/notes.txt
```

```
(On Windows:)
users\alice\documents\report.txt
users\bob\notes.txt
```

What is happening?

- `path.join()` intelligently merges segments. It removes duplicate separators and resolves `..` (parent directory) references.
- The output automatically uses `/` or `\` depending on the operating system.

5.2 `path.resolve()` — Getting Absolute Paths

While `path.join()` produces relative paths, `path.resolve()` always returns an **absolute path**. It works right to left, treating each argument as a directory until it constructs a complete absolute path:

```
path-resolve.js
```

```
const path = require('path');

// Resolves relative to the current working directory
const abs1 = path.resolve('data', 'users.json');
console.log('Resolve relative:', abs1);

// An absolute path argument resets the resolution
const abs2 = path.resolve('/tmp', 'logs', 'app.log');
console.log('Resolve absolute:', abs2);

// __dirname gives the directory of the current script file
console.log('__dirname:', __dirname);
```

```
const configPath = path.resolve(__dirname, 'config', 'settings.json');
console.log('Config path:', configPath);
```

```
node path-resolve.js
```

Output:

```
Resolve relative: /home/student/fs-basics/data/users.json
Resolve absolute: /tmp/logs/app.log
__dirname: /home/student/fs-basics
Config path: /home/student/fs-basics/config/settings.json
```

Key Insight: Use `path.resolve(__dirname, ...)` when you need a path relative to the *script file's location*, not the directory from which `node` was invoked. This is critical — if you run `node scripts/read.js` from the project root, `path.resolve('data.txt')` resolves relative to the project root, not to the `scripts/` folder. Using `__dirname` removes this ambiguity.

5.3 `path.basename()`, `path.dirname()`, `path.extname()`

These methods extract specific parts from a path string:

```
path-parts.js
```

```
const path = require('path');

const filePath = '/home/student/projects/app/server.js';

console.log('Full path: ', filePath);
console.log('basename: ', path.basename(filePath));           // server.js
console.log('basename (no ext):', path.basename(filePath, '.js')); // server
console.log('dirname: ', path.dirname(filePath));              // /home/student/projects/app
console.log('extname: ', path.extname(filePath));              // .js

// Works with any file extension
console.log(path.extname('photo.jpeg'));    // .jpeg
console.log(path.extname('archive.tar.gz')); // .gz
console.log(path.extname('Makefile'));       // (empty string)
```

```
node path-parts.js
```

Output:

```
Full path: /home/student/projects/app/server.js
basename: server.js
basename (no ext): server
dirname: /home/student/projects/app
extname: .js
.jpeg
.gz
```

What is happening?

- `path.basename()` returns the last segment of the path — the file name. Passing a second argument strips that extension.
- `path.dirname()` returns everything *except* the last segment — the containing directory.
- `path.extname()` returns the file extension (including the dot). For files with no extension, it returns an empty string.

5.4 `path.parse()` — All Parts at Once

```
path-parse.js
```

```
const path = require('path');

const parsed = path.parse('/home/student/projects/app/server.js');
console.log(parsed);
```

```
node path-parse.js
```

Output:

```
{
  root: '/',
  dir: '/home/student/projects/app',
  base: 'server.js',
  ext: '.js',
  name: 'server'
}
```

`path.parse()` returns an object with all components at once. This is useful when you need multiple parts of the path and want to avoid calling three separate methods.

Warning: Never build file paths using string concatenation (`'data/' + filename`). This approach produces incorrect paths on Windows, does not handle edge cases like double slashes or `..` segments, and is a source of path traversal security vulnerabilities. Always use `path.join()` or `path.resolve()` .

6. Directory Operations

6.1 Creating Directories — `mkdir`

`mkdir-demo.js`

```
const fs = require('fs/promises');

async function createDirs() {
  // Create a single directory
  await fs.mkdir('logs');
  console.log('Created: logs/');

  // Create nested directories with { recursive: true }
  await fs.mkdir('data/users/profiles', { recursive: true });
  console.log('Created: data/users/profiles/');
}

createDirs();
```

`node mkdir-demo.js`

Output:

```
Created: logs/
Created: data/users/profiles/
```

What is happening?

- `fs.mkdir('logs')` creates a single directory. If `logs/` already exists, this call throws an `EEXIST` error.
- `{ recursive: true }` creates the entire directory tree. If `data/` and `data/users/` do not exist, they are created automatically. This option also prevents errors if the directory already exists.

The resulting folder structure:

```
fs-basics/  
├─ logs/  
├─ data/  
│   └─ users/  
│       └─ profiles/  
├─ message.txt  
├─ mkdir-demo.js  
└─ package.json
```

6.2 Listing Directory Contents — `readdir`

`readdir-demo.js`

```
const fs = require('fs/promises');  
const path = require('path');  
  
async function listContents() {  
  // Basic listing – returns an array of file/folder names  
  const items = await fs.readdir('.');  
  console.log('Items in current directory:', items);  
  
  console.log('---');  
  
  // With { withFileTypes: true } – returns Dirent objects  
  const entries = await fs.readdir('.', { withFileTypes: true });  
  
  for (const entry of entries) {  
    const type = entry.isDirectory() ? 'DIR' : 'FILE';  
    console.log(` ${type} ${entry.name}`);  
  }  
}
```



```
}  
  
listContents();
```

```
node readdir-demo.js
```

Output:

```
Items in current directory: [ 'data', 'logs', 'message.txt', 'mkdir-demo.js', 'node_modules', 'output.txt',  
'package.json', 'readdir-demo.js' ]  
---  
DIR    data  
DIR    logs  
FILE   message.txt  
FILE   mkdir-demo.js  
DIR    node_modules  
FILE   output.txt  
FILE   package.json  
FILE   readdir-demo.js
```

What is happening?

- By default, `readdir()` returns an array of strings — just the names, with no indication of whether each entry is a file or directory.
- Passing `{ withFileTypes: true }` returns `Dirent` objects instead. Each `Dirent` has methods like `.isDirectory()`, `.isFile()`, and `.isSymbolicLink()`.

6.3 Removing Directories — `rmdir` and `rm`

```
rmdir-demo.js
```

```
const fs = require('fs/promises');  
  
async function cleanup() {  
  // rmdir only works on EMPTY directories  
  await fs.rmdir('logs');  
  console.log('Removed empty directory: logs/');  
  
  // To remove a directory and all its contents, use rm with recursive  
  await fs.rm('data', { recursive: true });  
  console.log('Removed directory tree: data/');
```

```
}  
  
cleanup();
```

```
node rmdir-demo.js
```

Output:

```
Removed empty directory: logs/  
Removed directory tree: data/
```

Important: `fs.rmdir()` only removes *empty* directories. If the directory contains files or subdirectories, it throws an `ENOTEMPTY` error. For non-empty directories, use `fs.rm(path, { recursive: true })` instead. Be extremely careful with recursive removal — there is no “undo”. Double-check the path before calling it.

Cross-platform equivalents in the terminal:

Operation	macOS / Linux	Windows (CMD)
Remove empty directory	<code>rmdir logs</code>	<code>rmdir logs</code>
Remove directory tree	<code>rm -rf data</code>	<code>rmdir /s /q data</code>

7. Handling File System Errors

File system operations fail for many reasons: the file does not exist, the user lacks permission, the disk is full, or the path is invalid. Node.js communicates these failures through error objects with a `code` property:

Error Code	Meaning	Common Cause
<code>ENOENT</code>	No such file or directory	

Error Code	Meaning	Common Cause
		Trying to read a file that does not exist
<code>EEXIST</code>	File or directory already exists	Calling <code>mkdir()</code> without <code>recursive: true</code>
<code>EACCES</code>	Permission denied	Insufficient file system permissions
<code>EISDIR</code>	Is a directory	Trying to read a directory as a file
<code>ENOTEMPTY</code>	Directory not empty	Calling <code>rmdir()</code> on a non-empty directory

7.1 Handling Errors Gracefully

`error-handling.js`

```
const fs = require('fs/promises');

async function safeRead(filePath) {
  try {
    const content = await fs.readFile(filePath, 'utf8');
    console.log('Contents:', content);
  } catch (err) {
    switch (err.code) {
      case 'ENOENT':
        console.error(`File not found: ${filePath}`);
        break;
      case 'EACCES':
        console.error(`Permission denied: ${filePath}`);
        break;
      case 'EISDIR':
        console.error(`Cannot read a directory as a file: ${filePath}`);
        break;
      default:
        console.error(`Unexpected error (${err.code}): ${err.message}`);
    }
  }
}
```

```
// Test with a file that does not exist
safeRead('nonexistent.txt');
```

```
node error-handling.js
```

Output:

File not found: nonexistent.txt

7.2 Checking if a File Exists Before Operating

Rather than catching `ENOENT`, you can check for existence first using `fs.access()`:

```
check-exists.js
```

```
const fs = require('fs/promises');
const { constants } = require('fs');

async function readIfExists(filePath) {
  try {
    // Check if the file exists and is readable
    await fs.access(filePath, constants.R_OK);
    const content = await fs.readFile(filePath, 'utf8');
    console.log('Contents:', content);
  } catch {
    console.log(`${filePath} does not exist or is not readable`);
  }
}

readIfExists('message.txt');
readIfExists('ghost.txt');
```

```
node check-exists.js
```

Output:

Contents: Hello from the file system.
This is line two.
And this is line three.
ghost.txt does not exist or is not readable

Note: There is a known pattern called TOCTOU (*time-of-check to time-of-use*) — the file could be deleted between the `access()` check and the `readFile()` call. In production code, it is often better to simply attempt the operation and handle the error, rather than checking first. The `try/catch` approach in Section 7.1 is generally preferred.

8. Combining `fs` and `path` — A Practical Example

The `fs` and `path` modules are almost always used together. Here is a realistic script that organises files in a directory by extension:

`organise-by-ext.js`

```
const fs = require('fs/promises');
const path = require('path');

async function organiseByExtension(dirPath) {
  // Read all entries in the target directory
  const entries = await fs.readdir(dirPath, { withFileTypes: true });

  for (const entry of entries) {
    // Skip directories – only process files
    if (!entry.isFile()) continue;

    const ext = path.extname(entry.name).slice(1); // remove the dot
    if (!ext) continue; // skip files without extensions

    // Create a subdirectory for this extension if it doesn't exist
    const targetDir = path.join(dirPath, ext);
    await fs.mkdir(targetDir, { recursive: true });

    // Move the file into the subdirectory
    const oldPath = path.join(dirPath, entry.name);
    const newPath = path.join(targetDir, entry.name);
    await fs.rename(oldPath, newPath);

    console.log(`Moved: ${entry.name} -> ${ext}/`);
  }
}
```

```
    console.log('Organisation complete');
  }

  // Create test files first, then organise
  async function demo() {
    const testDir = path.join(__dirname, 'messy-folder');
    await fs.mkdir(testDir, { recursive: true });

    // Create some test files
    await fs.writeFile(path.join(testDir, 'notes.txt'), 'Some notes');
    await fs.writeFile(path.join(testDir, 'todo.txt'), 'Buy milk');
    await fs.writeFile(path.join(testDir, 'app.js'), 'console.log("hi")');
    await fs.writeFile(path.join(testDir, 'utils.js'), 'module.exports = {}');
    await fs.writeFile(path.join(testDir, 'style.css'), 'body { margin: 0 }');

    console.log('Test files created');
    await organiseByExtension(testDir);
  }

  demo();
```

```
node organise-by-ext.js
```

Output:

```
Test files created
Moved: app.js -> js/
Moved: notes.txt -> txt/
Moved: style.css -> css/
Moved: todo.txt -> txt/
Moved: utils.js -> js/
Organisation complete
```

The resulting structure:

```
messy-folder/
├─ css/
│   └─ style.css
├─ js/
│   ├── app.js
│   └─ utils.js
```

```
└─ txt/
   └─ notes.txt
   └─ todo.txt
```

What is happening?

- `readdir()` with `withFileTypes` lets us distinguish files from directories.
- `path.extname()` extracts the extension. `.slice(1)` removes the leading dot (`.js` becomes `js`).
- `mkdir()` with `recursive: true` safely creates the target folder (no error if it already exists).
- `fs.rename()` moves a file from one path to another. This is an atomic operation on most file systems.

Summary

Concept	Key Point
<code>fs</code> module	Core module for all file system operations. Three flavours: sync, callback, and promise (<code>fs/promises</code>).
<code>readFile</code> / <code>readFileSync</code>	Reads entire file into memory. Pass encoding (<code>'utf8'</code>) for text files, omit for binary data (returns Buffer).
<code>writeFile</code>	Creates a new file or replaces existing content entirely. Destructive operation.
<code>appendFile</code>	Adds content to the end of a file. Creates the file if it does not exist. Non-destructive.
<code>path.join()</code>	Combines path segments with the platform-correct separator. Handles normalisation.
<code>path.resolve()</code>	Returns an absolute path. Use with <code>__dirname</code> for script-relative paths.

Concept	Key Point
<code>path.basename/dirname/extname</code>	Extract filename, directory, or extension from a path string.
<code>mkdir</code>	Creates directories. Use <code>{ recursive: true }</code> for nested creation and to avoid <code>EEXIST</code> errors.
<code>readdir</code>	Lists directory contents. Use <code>{ withFileTypes: true }</code> to get <code>DirEntry</code> objects with <code>.isFile()</code> / <code>.isDirectory()</code> .
<code>rmdir</code> / <code>rm</code>	<code>rmdir</code> removes empty directories. <code>rm({ recursive: true })</code> removes directories and all contents.
Error codes	<code>ENOENT</code> (not found), <code>EEXIST</code> (already exists), <code>EACCES</code> (permission denied), <code>ENOTEMPTY</code> (directory not empty).

Interview Questions

1. What is the difference between `fs.readFile()` and `fs.readFileSync()`, and when would you use each?

Expected answer: `readFileSync` blocks the event loop until the read completes and returns the data directly. `readFile` is asynchronous — it returns immediately and invokes a callback (or resolves a promise) when the data is ready. Use the synchronous variant only during application startup or in CLI scripts where blocking is acceptable. In a server handling concurrent requests, always use the asynchronous variant.

2. What happens if you call `readFileSync()` without passing an encoding argument?

Expected answer: It returns a `Buffer` object containing the raw binary data instead of a string. To get a string, pass `'utf8'` as the second argument or call `.toString('utf8')` on the Buffer.

3. What is the difference between `writeFile` and `appendFile` ?

Expected answer: `writeFile` creates the file or replaces its entire content if it already exists. `appendFile` adds data to the end of the file without removing existing content, and creates the file if it does not exist.

4. Why should you use `path.join()` instead of string concatenation to build file paths?

Expected answer: `path.join()` uses the correct separator for the operating system (`/` on macOS/Linux, `\` on Windows), normalises redundant separators and `..` segments, and avoids path traversal issues. String concatenation produces platform-dependent, error-prone paths.

5. What is the difference between `path.join()` and `path.resolve()` ?

Expected answer: `path.join()` simply concatenates path segments with the correct separator and returns a relative or absolute path depending on the input. `path.resolve()` always returns an absolute path, resolving segments from right to left and using the current working directory as a base when no absolute segment is found.

6. What does the `ENOENT` error code mean, and how would you handle it?

Expected answer: `ENOENT` stands for "Error: No Entity" and means the file or directory does not exist. Handle it by catching the error and checking `err.code === 'ENOENT'`, then either creating the file, returning a default value, or logging a meaningful message.

7. How do you create a deeply nested directory structure (e.g. `a/b/c/d`) in one call?

Expected answer: Pass `{ recursive: true }` as the options argument to `fs.mkdir()`. Without this option, `mkdir` only creates a single directory and throws `ENOENT` if parent directories are missing.

8. What is the TOCTOU problem in the context of file system operations?

Expected answer: TOCTOU (time-of-check to time-of-use) occurs when you check for a condition (e.g., file exists) and then act on it, but the condition changes between the check and the action. For example, checking if a file exists with `access()` and then reading it — another process could delete the file in between. The preferred pattern is to attempt the operation directly and handle the error if it fails.

Practical Assignment: Build a Note Manager

Objective: Build a command-line note manager that creates, reads, lists, and deletes plain-text notes, storing each note as a separate file inside a `notes/` directory.

Setup:

1. Create a project folder called `note-manager` and initialise it with `npm init -y`.
2. Create a file called `notes.js`.

Requirements:

1. The script accepts a command and arguments via `process.argv` :
 - `node notes.js add "shopping-list" "Milk, eggs, bread"` — Creates `notes/shopping-list.txt` with the given content.
 - `node notes.js read "shopping-list"` — Reads and displays the content of `notes/shopping-list.txt`.
 - `node notes.js list` — Lists all `.txt` files in the `notes/` directory, showing name and file size.
 - `node notes.js remove "shopping-list"` — Deletes `notes/shopping-list.txt`.
2. Use `fs/promises` with `async/await` for all file operations.
3. Use `path.join()` to construct all file paths.
4. Ensure the `notes/` directory is created automatically if it does not exist (hint: `mkdir` with `recursive: true`).
5. Handle errors gracefully — reading a non-existent note should print a friendly message, not a stack trace.

Expected output:

```
node notes.js add "shopping-list" "Milk, eggs, bread"
```

Output:

```
Note saved: notes/shopping-list.txt
```

```
node notes.js add "todo" "Finish fs module exercises"
```

Output:

Note saved: notes/todo.txt

```
node notes.js list
```

Output:

Notes:
shopping-list.txt (19 bytes)
todo.txt (30 bytes)

```
node notes.js read "shopping-list"
```

Output:

--- shopping-list ---
Milk, eggs, bread

```
node notes.js read "nonexistent"
```

Output:

Note not found: nonexistent

```
node notes.js remove "shopping-list"
```

Output:

Note removed: shopping-list

Bonus Challenge

Add an `append` command that adds text to an existing note instead of replacing it:

```
node notes.js append "todo" "Review path module notes"
```

Output:

Appended to: notes/todo.txt

Also add a `stats` command that shows the total number of notes and the combined size of all note files. Hint: use `fs.stat()` to get file metadata including `size`.

